MEMPIRE

Verification Report

Full-product test plan, executed against the live system

VERDICT: 91 ITEMS PASS · 1 RETIRED · 0 FAIL

Product	play.mempire.fun — a real-time Solana card battler
Roster	36 verified assets: majors, memecoins and tokenised stocks
Method	real browser sessions, real signed devnet transactions, two independent clients
Test record	TESTPLAN.md, runs 1-11 (github.com/nickthelegend/mempire)
Report date	August 20, 2026

Produced by an automated verification agent (Claude). Every claim in this report is backed by an on-chain signature, a recorded browser session, or a reproducible script in the repository.

1 · Executive summary

Mempire is a card battler whose roster is the market itself — memecoins, majors and tokenised stocks, each one a fighter. It never custodies, locks or stakes a player's holdings: the coins are characters, not collateral, and the only thing anyone risks is the SOL they choose to put on a match. It was tested the way a hostile user would use it: through the real deployed product, with real money movements on Solana devnet, against a written plan that defines what “correct” means for every screen, endpoint, instruction, and integration. The plan reached **92 items**; after eleven runs of test-fix-retest, **every active item is a verified PASS and none fail**. One item (C4) tested the staking subsystem, which has since been deliberately removed — every held token was returned to its owner first, and the instructions’ absence from the live program was then verified. Verification was re-confirmed against the live system on the date of this report: a fresh browser session touring every screen produces **zero console errors and zero failed network requests**.

Metric	Result
Plan items verified	**91 / 91 active items PASS — 0 FAIL**
Retired since the plan was written	1 (C4, staking) — the capability was deliberately removed, and its absence verified
Defects found and fixed at root cause	34 across ten runs (catalog in §4)
Mainnet-readiness audit (62 parallel reviewers)	41 confirmed gaps — all closed in code or fenced with documented reasons
Mocks / stubs / fallback data in tested surface	None. Every path hits the real chain, relay, and database.
Console & network, fresh session, all screens	0 errors, 0 failed requests (2 informational log lines)
Real on-chain evidence	40+ signed devnet transactions cited in §5

2 · Method

Real product, not the code. All flows were driven through the deployed app at play.mempire.fun in a real Chromium browser — clicks, drags onto the battle canvas, wallet flows — with the console and network recorder open for every item. **Real money paths.** Match wagers, mints, merges, swaps and escrows were executed as signed transactions against the deployed devnet programs and confirmed by reading the chain back, never by trusting the UI. **Two independent clients.** PvP settlement was proven by running a second, fully independent client (real deck, real escrow, its own simulation of the match) so that settlement required two parties who never trusted each other to agree. **Adversarial audit.** The mainnet-readiness pass swept the entire codebase with 62 parallel reviewers; every blocker-level claim was independently re-verified against the exact file and line before being accepted.

3 · Results by section

A · Product flows (screens, matchmaking, cards, deck, clans, empire, swap, battle)

Items	What was verified	Status
A1.1–A1.9	Arena: load, guest connect, tier gating, queueing, bot fallback, reload mid-queue	PASS
A2.1–A2.17	Cards: starter kit, mint (no holdings required), re-mint guard, merge-to-level, max-level guard, shop, chests, deep scroll	PASS
A3.1–A3.5	Deck: slots, one-per-coin rule, power recompute, persistence	PASS
A4.1–A4.6	Clans: browse, create, insufficient funds, join/leave, crowns reporting	PASS
A5.1–A5.5	Empire: balances, history, stranded-stake recovery from chain	PASS
A6.1–A6.5	Swap: live pool state, quote maths vs reserves, execution, guards, junk input	PASS
A7.1–A7.7	Battle: canvas, deploy, illegal drops, elixir economy, combat, honest results, forfeit	PASS

B · API & relay (14 routes + matchmaker WebSocket)

Items	What was verified	Status
B1–B12	Health, coins, faucet (+replay guard), player, leaderboard, ladder, clans, telemetry, analytics	PASS
B13	Match ingest — unsigned writes 401; money facts read from the chain, not the client	PASS
B14	Matchmaker — signed queues pair ranked; unsigned demote to casual	PASS

C · On-chain programs (mempire, rollup, AMM on devnet)

Items	What was verified	Status
C1–C3	mint_card, upgrade_card, tokenize_card (real Metaplex NFT, authority burned)	PASS
C4	Staking retired — every held token returned first, then the instructions removed and verified gone from the live program	PASS
C5	create_match / join_match / settle_from_log — escrow held, two independent sims agreed, winner paid	PASS
C6	claim_timeout / cancel_match — stranded stakes recovered with correct refunds	PASS
C7	Ephemeral Rollup round trip — delegate, plays land on rollup, state hashes, commit home	PASS
C8	Chests — two-claim entitlement, MagicBlock VRF roll, honest local-roll labeling	PASS
C9	AMM swap — constant product with 30 bps fee, reserves moved by the predicted amount	PASS

D · External integrations

Items	What was verified	Status
D1–D3	Devnet RPC backoff; MagicBlock router placement; delegated writes on the returned rollup	PASS
D4–D6	Metaplex metadata (64/64 serve 200), DexScreener boundary, Mongo persistence across restarts	PASS

E · Quality bars

Items	What was verified	Status
E1–E2	Zero console errors, zero unexpected 4xx/5xx — re-confirmed on report date	PASS
E3–E6	Desktop & mobile layout, reload mid-flow, fully playable with no wallet installed	PASS
E7–E8	No mocks/stubs anywhere in a tested path; all interactive controls accessibly named	PASS

4 · Notable defects found and fixed

Thirty-four defects were fixed at root cause across the ten runs. The ones that mattered most:

Defect	Impact	Fix
Deck collapse	Minting one card left a 1-card deck and locked every game mode	Chain and seeded collections merged; minted cards promoted
Fake stake	Staking an unminted card wrote “Staked $75” with no transaction — inflating matchmaking power	Onchain sessions stake onchain or state why they cannot
Stranded unstake	After any refresh the Claim button vanished forever; 20 BTC sat unreachable in the vault	Fixed, then the whole subsystem retired — every token drained back to its owner first
Guest auth split-brain	Guest signed as one identity while claiming another; every write silently 401’d	The signer's derived address is the claimed address, always
Frozen match	A connected-but-silent opponent froze the clock forever; forfeiting was the only exit	Stalls resolve like disconnects: run out the sim, take the real result
Lost crowns	Crown reports went unsigned to a signed route AND required a screen most winners never open	Resolve the clan, sign the report; verified 2→3→5 automatically
First-caller chest theft	Whoever called end_log first declared the winner and took the chest	Two-claim settlement; grant fires only at agreeing pair-completion (observed on-chain)
Leaderboard poisoning	The board believed client-asserted payouts; one crafted POST faked riches	Relay reads the settled match account; fabricated claims rank as zero (tested live)
Scam-token registry	Symbol search returned a “BTC” with $8B of spoofed liquidity	Identity only from Jupiter's verified list; market data per-mint; decimals RPC-checked
Public treasury key	The fee destination derived from a public string in the repository	Script refuses mainnet; runbook mandates a fresh key or multisig

5 · On-chain evidence (selected)

Devnet, program BnLDCAREDpBGenqZr8BTyQu7BCoVewF9XEtMPFBqFxeP (+ rollup, AMM). Signatures truncated for print; all are on the public ledger.

What it proves	Evidence
Staking fully retired	`19 held cards drained to zero, then the instructions removed and redeployed (5aY78HwPKNgMpBbz…). Calling request_unstake on the live program now fails: the instruction does not exist.`
Happy-path settlement	`Match #71: CreateMatch 48v7UxFpbPbW68… · JoinMatch 4gjY253hKES9pQ… · InitMatchLog 5BTFarDErVPKbY… · SettleFromLog 5GLMmqLLoZ1WZD… with claims [0,0] from two independent simulations`
Rollup lifecycle	`Match #72/#73: ROLLUP LIVE (38 plays sealed), 86 state hashes committed, log undelegated home, pot PAID`
Two-claim chest grant	`Rollup log #75: first claim held ([3,0], no grant) → agreeing claim 4CR7k5YCnaW1… granted at pair-completion (earned 1→2) and sealed [0,0]`
Dispute safety	`Base log #75 claims [2,0] (genuine divergence) → no winner paid, no chest granted, stakes recovered by timeout`
Max-level guard	`9 merges to level 10 (13,665 → 9,165 $MEMPIRE = exactly 100+200+…+900); 11th merge refused with MaxLevel, no fee taken`
NFT tokenization	`tokenize_card → supply-1, 0-decimal Metaplex NFT, mint authority burned; renders in Solana Explorer`
AMM swap	`Swap 5N1woW2Wi6qDCb1RxE…: reserves 1,705,532.45 → 1,706,532.45 $MEMPIRE / 19.00 → 18.9889 USDC — the exact constant-product prediction`
Replay guard (live relay)	`Identical signed request twice: 200 then 401 “signature already used”`
Timeout recovery	`ClaimTimeout 3Ep5LqW8fAkRV5VDf5… / 5SZ9NxfqFEhE22PByd… · ReleaseCards DZgsLiPeibWw37… / 4TjMuqjcaF3mm6… (16 locked cards freed)`

6 · Mainnet readiness and cost

A dedicated audit (62 parallel reviewers, every blocker independently re-verified) found 41 confirmed gaps between the devnet build and a mainnet launch. All are closed: one env switch decides the cluster and everything derives from it; a build confused about its chain refuses to boot; guest keys can never sign on mainnet; the roster is built from Jupiter-verified identities with live market data; priority fees attach to every mainnet transaction; the faucet cannot exist on a mainnet relay.

Deploy cost is a size decision, and the two heaviest subsystems are now cargo features rather than fixed weight. Measured from this repository's own builds:

Build	Bytes	SOL	Adds
lean — the launch build	463,456	**3.23**	the whole game: roster, decks, chests, escrowed PvP, settlement
+ nft	594,224	4.14	cards as Metaplex 1-of-1s
+ rollup	574,624	4.00	MagicBlock ER, VRF chests, play log
both — what devnet runs	704,432	4.90	everything the plan above verified

Settlement is identical in every one of them — the two seats' claims are the same bytes whether the log visits a rollup or stays on base layer — so the deferred features cost their difference later, via `solana program extend`, rather than upfront. And the rent is a **refundable deposit, not a spend**: closing the program returns it. Launch is ≈3.4 SOL all-in, of which roughly 0.05 is actually consumed. Runbook in MAINNET.md.

7 · Honest limits

This report claims exactly what was tested, and no more. (1) All money-path verification ran on **devnet**; mainnet execution awaits funding and repeats the same suite via the runbook. (2) The programs are **unaudited by a third-party firm**; compensating controls are small stake caps and a dispute design where cheating voids rather than pays — a documented residual lets a sore loser force a refund (never a theft). (3) One deliberately human-gated external (Circle's USDC faucet) was bypassed by verifying the swap in the opposite direction with real tokens. (4) The **lean launch build is measured but not yet separately re-verified end to end**: the plan above ran against the full build, and the lean one differs only by compiling out the NFT mint and the rollup trip. Its settlement path — claims written on base layer — is the same code the full build already runs whenever a log is not delegated. (5) Two automation-environment artifacts (hidden-tab WebSocket reaping, zero-size viewports) affected the test harness, not the product, and are recorded in the test log.

ATTESTATION

Every active item of the test plan in TESTPLAN.md holds a verified PASS against the live system as of August 20, 2026 — 91 pass, none fail, and the one retired item (C4, staking) was verified absent after its tokens were returned. Zero mocks, zero stubs, and zero fallback data stand in for a real dependency anywhere in the tested surface. A fresh browser session touring every screen produces zero console errors and zero failed network requests. Where a claim rests on money movement, the evidence is a signed transaction on the public devnet ledger, cited above and reproducible from the repository's scripts.

Produced by Claude, an automated verification agent, from recorded browser sessions, on-chain reads, and the repository's own test log. This is an engineering verification record, not a third-party security audit.